

Modelos de decisión: qué son y para qué sirven

Una introducción para quien trabaja con texto administrativo

Joaquín Herrero asistido por Claude Opus 5.5

Septiembre de 2026

Qué son

Un **modelo de decisión** es un modelo de inteligencia artificial que no escribe. Recibe un texto, una pregunta cerrada y una lista de respuestas posibles, y devuelve **una probabilidad para cada respuesta**.

Un ejemplo:

Texto: «El interesado presentó la solicitud el 15 de enero, pero no consta el pago de la tasa.»

Pregunta: ¿Está la solicitud completa?

Respuesta del modelo: sí 3 % · no 91 % · no consta 6 %

El modelo no redacta una explicación ni puede contestar algo que no esté en la lista. Solo reparte la probabilidad entre las opciones que se le dan.

Admiten tres tipos de pregunta:

Tipo	Qué devuelve	Ejemplo
Elegir una opción	Probabilidad de cada opción	¿Qué tipo de escrito es: solicitud, alegación, recurso u otro?
Puntuar	Probabilidad de cada punto de una escala	Del 1 al 5, ¿qué urgencia tiene?
Sí o no	Probabilidad de sí, de no y de «no consta»	¿Menciona el texto un plazo?

A este tipo de modelo se le ha llamado también «Sistema 1», por la distinción de Daniel Kahneman entre el pensamiento rápido e intuitivo (Sistema 1) y el lento y deliberado (Sistema 2). Un modelo de decisión hace juicios rápidos sobre un texto; no razona en varios pasos ni planifica.

Si no usan un *prompt*, ¿cómo se les pregunta?

Con un chat se escribe un *prompt*: un texto libre que mezcla instrucciones («actúa como...», «responde solo con una palabra»), el contenido y, a menudo, el formato que se espera en la respuesta. Afinar esa redacción es un oficio en sí mismo, y un cambio pequeño en el *prompt* puede cambiar la respuesta.

A un modelo de decisión no se le escribe un *prompt* libre, sino que **se rellena un formulario con campos fijos**, igual que se llama a cualquier otro servicio desde una aplicación:

Campo	Qué contiene	Ejemplo
Texto	El contenido sobre el que se decide	El escrito recibido
Pregunta	La pregunta, en lenguaje natural	¿Está la solicitud completa?
Tipo	Elegir una opción, puntuar o sí/no	Elegir una opción
Opciones	Las respuestas posibles, cada una con una descripción breve si hace falta	sí · no · no consta

Sí hay lenguaje natural (el texto, la pregunta y las opciones están en castellano), pero no hay instrucciones sobre cómo comportarse ni sobre cómo dar formato a la respuesta: eso lo fija el propio modelo, que siempre devuelve lo mismo, una probabilidad por opción. Por eso no hay que «convencerlo» de nada ni pelearse con respuestas que se salen del guion.

Dos detalles prácticos:

- **Se pueden hacer varias preguntas sobre el mismo texto en una sola llamada.** El modelo lee el texto una vez y responde cada pregunta por separado, sin que la respuesta a una influya en las demás.
- **Las opciones son parte de la pregunta.** Escribirlas bien (que no se solapen, que cubran todos los casos, que haya un «no consta» cuando el texto puede no decir nada) es lo que más influye en la calidad del resultado. Es el equivalente a redactar bien un *prompt*, pero mucho más acotado.

En qué se diferencian de un chat como ChatGPT

Los modelos de lenguaje más conocidos (ChatGPT, Claude, Gemini y similares) **generan texto**. Un modelo de decisión **elige entre opciones cerradas**. De esa diferencia salen casi todas sus ventajas:

	Modelo de lenguaje (chat)	Modelo de decisión
Qué devuelve	Texto libre	Una probabilidad por opción
¿Puede inventar?	Sí, puede dar una respuesta falsa bien redactada	No puede salirse de las opciones
Integración en una aplicación	Hay que interpretar el texto que devuelve	Devuelve datos, directamente utilizables
Coste y tiempo por consulta	Alto: segundos por respuesta	Bajo: décimas de segundo
Tamaño	Grande; los mejores solo en la nube	Pequeño; los hay que caben en un servidor corriente
Medición	Difícil: hay que valorar textos	Sencilla: se cuenta cuántas veces acierta

La consecuencia práctica: para **clasificar, encaminar o comprobar**, un modelo de decisión es más barato, más rápido y más fácil de controlar que un chat. Para **redactar, resumir o explicar**, no sirve.

¿Y cómo se les entrena para que sepan responder?

No se construyen desde cero. El proceso, tal como lo documentan los modelos abiertos (el de Jev no es público), tiene tres pasos:

1. Se parte de un modelo de lenguaje ya entrenado. Un modelo de lenguaje ha leído enormes cantidades de texto y ya «entiende» el idioma: sabe qué es una negación, un plazo o una fecha. Esa comprensión es lo que se aprovecha.

2. Se le cambia la salida. En lugar de la pieza que escribe la siguiente palabra, se le añade una pieza pequeña (la «cabeza») que mira cada una de las opciones y les da una puntuación. Esas puntuaciones se convierten en probabilidades que suman 100 %. Desde ese momento el modelo ya no puede escribir: solo puede elegir.

3. Se le entrena con muchísimos ejemplos resueltos. Cada ejemplo es un texto, una pregunta, unas opciones y la respuesta correcta. Salen de colecciones públicas (clasificación de noticias, de reseñas, de consultas, preguntas de examen, razonamiento lógico...) y de ejemplos generados con reglas, en los que la respuesta correcta se conoce por construcción (por ejemplo, calcular si una fecha cae dentro de un plazo). En el entrenamiento, el modelo responde, se compara con la respuesta correcta y se corrige, ejemplo a ejemplo. La corrección castiga sobre todo **equivocarse con seguridad**: dar un 95 % a una opción falsa cuesta mucho más que darle un 55 %. Así aprende a repartir bien la probabilidad, no solo a acertar.

Además, durante el entrenamiento se toman precauciones para que aprenda la tarea y no atajos:

- **Se baraja el orden de las opciones**, para que no aprenda que «la buena suele ser la primera».
- **Se incluyen casos en los que la respuesta no está en el texto**, para que aprenda a decir «no consta» en lugar de adivinar.
- **Se reserva una parte de los ejemplos que el modelo nunca ve al entrenar**, para medir con ellos cuánto acierta de verdad y para ajustar al final la calibración: si el modelo resulta demasiado seguro de sí mismo, se le corrige con un factor que suaviza sus probabilidades.

Todo esto lo hace quien publica el modelo. Lo que puede hacer una organización es el paso siguiente: adaptarlo a sus propios casos.

Ajuste fino: adaptarlo a los casos propios

Un modelo de decisión recién descargado sabe hacer juicios generales, pero no conoce los tipos de escrito, la terminología ni los criterios de una organización concreta. Se le pueden enseñar. A todo entrenamiento que se hace sobre un modelo ya entrenado se le llama **post-entrenamiento** (*post-training*); la forma que está al alcance de una organización es el **ajuste fino** (*fine-tuning*): seguir entrenándolo, con el mismo mecanismo del paso 3, pero con casos propios.

Tres ideas lo hacen viable:

- **No se reentrena el modelo entero.** Se congela y se le añaden unas piezas pequeñas que son lo único que cambia (la técnica se llama LoRA). Se entrena en torno al 1 % de los parámetros, lo que cabe en una máquina propia con una buena tarjeta gráfica y tarda de minutos a pocas horas.
- **Las respuestas correctas ya existen.** Cada decisión que una persona ha tomado y ha quedado registrada (el tipo asignado a un escrito, la unidad a la que se envió, si se pidió subsanación) es un ejemplo resuelto. No hay que etiquetar desde cero: hay que extraer y revisar.
- **Nada sale de la organización.** Datos, entrenamiento y modelo resultante se quedan en su infraestructura.

Un ejemplo completo: el tipo de escrito en el registro

1. La decisión y la pregunta. Se fija por escrito, igual que se usará después:

- Pregunta: *¿Qué tipo de escrito es?*
- Opciones: solicitud · alegación · recurso de alzada · recurso de reposición · consulta · otro.

2. Los datos. Se extraen de la base de datos, por ejemplo, 2.000 escritos de los últimos años con el tipo que les asignó el personal de registro. Antes de entrenar se revisa a mano una muestra (unos 100): siempre aparecen etiquetas dudosas o erróneas, y es mejor apartarlas que enseñarle al modelo los errores del pasado. También se mira cuántos hay de cada tipo: si una opción es muy rara (un 3 % de los casos, pongamos), el modelo tenderá a ignorarla y habrá que compensarlo, buscando más ejemplos de esa opción o dándole más peso al entrenar.

3. El formato. Cada caso se convierte en una línea de un fichero, con los mismos campos que se usan para preguntar y, además, la respuesta correcta (aquí partida en varias líneas para que se lea

mejor):

```
{"state": "... interpone recurso de alzada contra la resolución de ...",  
"questions": {"tipo": {"  
  "type": "choice",  
  "instructions": "¿Qué tipo de escrito es?",  
  "criteria": {"solicitud": null, "alegación": null,  
    "recurso de alzada": null, "recurso de reposición": null,  
    "consulta": null, "otro": null},  
  "label": "recurso de alzada"}}}}
```

4. El reparto. Los casos se dividen en tres grupos que no se mezclan nunca:

Grupo	Parte	Para qué
Entrenamiento	70 %	Lo único que el modelo ve al aprender
Calibración	15 %	Para corregir al final lo seguro que se muestra
Prueba	15 %	Para medir el resultado; no se toca hasta el final

El reparto se hace **por expediente**, no por documento: si dos escritos del mismo expediente quedan uno en entrenamiento y otro en prueba, el modelo «reconoce» el caso y la medida sale mejor de lo que es.

5. El entrenamiento. Se parte del modelo publicado y se entrena unas pocas pasadas sobre los casos propios, mezclados con una parte de los ejemplos generales con los que se entrenó originalmente, para que no olvide lo que ya sabía. Con un modelo abierto como Kev es un comando (simplificado; el real añade los datos de arquitectura del modelo de partida):

```
python -m kev.train --init_from jaredpalmer/kev-4b \  
  --data registro/entrenamiento.jsonl --suite evals/v7/decision-v7 \  
  --replay 500 --lr 2e-5 --epochs 2 --out modelos/registro-v1
```

Los ajustes (cuántas pasadas, a qué ritmo aprende) se fijan **antes** de ver ningún resultado, para no ir retocándolos hasta que la prueba salga bien por casualidad.

6. La recalibración. Tras el ajuste, el modelo suele volverse demasiado seguro. Con el grupo de calibración se calcula un factor que corrige sus probabilidades para que un 90 % vuelva a significar 9 aciertos de cada 10.

7. La medida. Con el grupo de prueba, y comparando siempre con el modelo sin ajustar:

- acierto **en cada tipo de escrito**, no solo el global (un modelo que dijera siempre «solicitud» podría acertar el 60 % y no servir para nada);
- calibración: si su seguridad se corresponde con su acierto;
- olvido: que siga respondiendo bien a preguntas generales que ya sabía responder.

8. La decisión. Si mejora de forma clara, se fija un umbral y se pasa a un piloto en el que el modelo propone el tipo y la persona de registro lo confirma o lo corrige. Esas correcciones son, a su vez,

nuevos ejemplos para el siguiente ajuste.

Un aviso útil: antes de ajustar, conviene probar el modelo **sin ajustar** con la misma pregunta y el grupo de prueba. A veces basta con formular bien las opciones, y el ajuste no compensa el trabajo.

Un caso medido: ¿en qué apartado del BOE se publica?

Para comprobar si todo esto funciona con textos administrativos reales, se ha seguido el proceso anterior con datos públicos del Boletín Oficial del Estado. Es una clasificación documental muy parecida a la del registro: leer un texto y decir de qué tipo es.

El código, los resultados y las instrucciones para repetir la prueba están en <https://github.com/joakinen/modelos-de-decision>.

La pregunta. *¿En qué apartado del BOE se publica este texto?* Las opciones son las siete secciones del sumario:

Apartado	Qué contiene
I	Disposiciones generales (leyes, reales decretos, órdenes de alcance general)
II.A	Nombramientos, situaciones e incidencias del personal
II.B	Oposiciones y concursos
III	Otras disposiciones (actos concretos: convenios, subvenciones, resoluciones)
IV	Administración de Justicia
V.A	Anuncios de contratación del sector público
V.B	Otros anuncios oficiales

Los datos. Se descargaron de la API de datos abiertos del BOE. La respuesta correcta de cada texto es la sección en que lo publicó el propio BOE, así que es fiable por construcción. Al modelo se le da solo el cuerpo del texto, sin el título, que a menudo delata la sección.

- **Entrenamiento:** 1.400 textos de enero a junio de 2026, 200 por apartado.
- **Prueba:** 350 textos de julio y agosto de 2026, 50 por apartado. El modelo ajustado no vio nunca este periodo.
- Como mucho tres textos casi idénticos por tipo (por ejemplo, los cambios diarios del euro), para que no dominen.

Resultados en los 350 textos de prueba (acierto medio por apartado):

Modelo	Acierto	Log-loss
Kev-0.8B, ajustado con los 1.400 textos	95 %	0,35
Réplica abierta de Jev de 2.000 millones de parámetros, sin ajustar	73 %	1,24
Kev-4B, sin ajustar	69 %	0,83
Modelo de chat general de 9.000 millones (Qwen3.5), sin ajustar	66 %	0,78
Réplica abierta de Jev de 800 millones, sin ajustar	64 %	1,10
Kev-0.8B, sin ajustar	49 %	1,32

El log-loss mide lo bien que el modelo reparte la probabilidad: cuanto más bajo, mejor. Castiga sobre todo equivocarse con seguridad.

Lo que enseña:

- **Con datos suficientes, el ajuste funciona.** El mismo modelo pasa del 49 % al 95 %. La mejora es de +45 puntos, con un intervalo de confianza del 95 % de +41 a +49: no es casualidad.
- **Sin ajustar, ningún modelo conoce las convenciones del BOE.** Ninguno reconoce más de 5 de las 50 disposiciones generales: las confunden con «otras disposiciones». Distinguir una norma de alcance general de un acto concreto no se deduce del texto; se aprende con ejemplos. El ajustado acierta 42 de 50.
- **Un modelo pequeño y ajustado supera a uno grande sin ajustar.** El ajustado tiene 800 millones de parámetros; el modelo general de 9.000 millones se queda en el 66 % y tarda unas trece veces más por texto que Kev-0.8B.
- **No olvida lo que ya sabía.** En un control con preguntas ajenas al BOE, el modelo ajustado responde igual que antes.
- **Es asequible en una máquina propia.** El ajuste se hizo en un ordenador de sobremesa (Mac mini con chip M4 Pro y 24 GB de memoria), sin enviar nada fuera. Lo que costó está en la tabla siguiente.

Lo que cuesta el ajuste (medido por el propio entrenador de Kev):

	Kev-0.8B
Tiempo de ajuste	3 h 29 min
Ejemplos propios	1.400, más 160 generales de repaso
Pasadas por los datos	3 (4.680 ejemplos procesados)
Memoria máxima	3,3 GB de GPU; 7,0 GB el proceso
Qué se entrena	En torno al 1 % de los parámetros (LoRA)

A eso hay que sumar reunir los datos: descargar los 1.400 textos del BOE tardó unos 27 minutos. El tiempo de ajuste crece en proporción a los ejemplos y a las pasadas, y con el tamaño del modelo.

Y lo que pasa con pocos datos. En otra prueba, con 100 ejemplos de los que solo 17 eran de la clase difícil, el ajuste no se distinguió del azar: el modelo aprendió a responder casi siempre la clase mayoritaria. La diferencia entre un caso y otro no está en el modelo ni en la máquina, sino en tener **unos cientos de ejemplos de cada opción**.

Límites de esta medida. Es un solo periodo de prueba (dos meses) y una sola pregunta. El modelo ajustado no se ha recalibrado: antes de fijar un umbral habría que hacerlo con un conjunto aparte. Y los textos del BOE están más normalizados que los escritos que llegan a un registro, que serían más variados.

Fuente de los datos: Agencia Estatal Boletín Oficial del Estado (boe.es), reutilizados según sus condiciones de datos abiertos.

Por qué importa la probabilidad

Lo más útil de estos modelos no es la respuesta, sino **cuánta seguridad tiene en ella**. Un modelo útil acierta con 95 % y duda con 55 %. Eso permite repartir el trabajo:

- Lo que el modelo decide con mucha seguridad se tramita de forma automática o se propone por defecto.
- Lo que decide con poca seguridad va a una persona.

Para que esto funcione, la probabilidad tiene que significar lo que dice: de todas las veces que el modelo dice «90 %», debe acertar unas 9 de cada 10. Eso se llama **calibración**, y hay que comprobarla con casos propios antes de fijar ningún umbral. Un modelo que falla con un 95 % de seguridad es peor que uno que acierta menos pero sabe cuándo duda.

Qué tareas hacen bien

Todo lo que se pueda formular como una pregunta cerrada sobre un texto:

- **Clasificar:** qué tipo de documento es, a qué materia pertenece.
- **Encaminar:** a qué unidad, cola o persona debe ir.
- **Comprobar:** si falta un documento, si se cumple una condición, si se menciona un requisito.
- **Detectar:** urgencia, reclamación, queja, plazo, datos personales.
- **Priorizar:** puntuar para ordenar una cola de trabajo.
- **Verificar a otro sistema:** comprobar si la respuesta que ha dado otro programa (o un chat) es correcta o está sustentada en el texto.
- **Abstenerse:** responder «no consta» cuando el texto no dice nada, en lugar de suponer.

Posibles usos en una administración

Algunos ejemplos, formulados como la pregunta que se le haría al modelo:

Uso	Pregunta	Opciones
Registro de entrada	¿Qué tipo de escrito es?	solicitud · alegación · recurso · consulta · otro
Reparto	¿Qué unidad debe tramitarlo?	la lista de unidades
Compleitud	¿Aporta la documentación obligatoria?	sí · no · no consta
Plazos	¿Se presentó dentro de plazo según el texto?	sí · no · no consta
Urgencia	¿Con qué urgencia hay que atenderlo?	escala del 1 al 5
Consultas	¿La respuesta a esta consulta está en la documentación publicada?	sí · no
Calidad	¿Cita la resolución la norma aplicable?	sí · no
Revisión	¿Contiene datos personales que haya que anonimizar?	sí · no · no consta

En todos ellos el modelo no sustituye a quien tramita: **ordena, filtra y señala**, y deja la decisión a una persona cuando hay duda o cuando la decisión tiene efectos sobre alguien.

También encajan bien **antes de un modelo de lenguaje**: el modelo de decisión, barato y rápido, clasifica todas las entradas, y solo las que lo necesitan pasan a un chat, más caro, para redactar un borrador.

Qué no hacen, y con qué cuidado usarlos

- **No redactan, no resumen, no explican.** Solo eligen entre opciones.
- **Dependen de cómo se formule la pregunta.** Opciones ambiguas o que se solapan dan resultados pobres. Formular bien la pregunta es la mitad del trabajo.
- **Fallan con la ironía, los dobles sentidos y los razonamientos largos** (contar días hábiles, encadenar varias condiciones). Ahí conviene que el modelo dude, y medir si lo hace.
- **Hay que medirlos con casos propios.** Las cifras de los fabricantes se obtienen con sus datos, no con los de quien los va a usar. Hacen falta casos ya resueltos por personas para saber cuánto acierta y si su probabilidad es fiable.
- **Adaptarlos a los casos propios tiene coste.** Se puede ajustar un modelo abierto con ejemplos propios, pero con pocos ejemplos aprende poco, y el ajuste puede estropear la calibración si no se vuelve a medir.
- **Las decisiones con efectos sobre personas requieren intervención humana.** El Reglamento General de Protección de Datos (artículo 22) limita las decisiones basadas únicamente en tratamiento automatizado, y la Ley 40/2015 (artículo 41) exige que la actuación administrativa automatizada tenga un órgano responsable definido. Usados como apoyo a la tramitación, estos límites se respetan con facilidad; usados para resolver, no.

Qué modelos existen

La idea la popularizó **Jev**, de la empresa TypeSafe AI (2026). Es un modelo cerrado: solo se puede usar como servicio en la nube, enviando los textos a sus servidores. Es muy barato por consulta, pero implica que los datos salen de la organización.

Poco después aparecieron **modelos abiertos** con el mismo planteamiento. Uno de ellos es **Kev**, con licencia Apache-2.0 y varios tamaños (de 0,8 a 27 mil millones de parámetros). Los pequeños funcionan en un servidor corriente o en un ordenador de sobremesa con buena memoria, sin conexión a internet. Eso permite:

- que **ningún dato salga** de la propia infraestructura;
- **no depender de un proveedor** ni de su precio;
- **ajustarlos con los propios casos**, con herramientas libres.

Son proyectos recientes y cambian rápido: conviene tratarlos como tecnología en evaluación, no como producto maduro.

Cómo se empezaría

Para una organización que quiera probarlo, el camino razonable es pequeño y medible:

1. **Elegir una sola decisión** concreta y frecuente, que hoy se tome a mano (por ejemplo, el tipo de escrito en el registro).
2. **Escribir la pregunta y sus opciones** exactamente como se usarían.
3. **Reunir casos ya resueltos por personas** (el ejemplo del registro, más arriba, detalla el proceso). Como orientación: 100 a 200 sirven para probar el proceso; 300 a 600 para un primer ajuste; 1.000 a 3.000 para algo que vaya a producción.
4. **Medir antes de crear:** acierto en cada opción (no solo el global) y calibración.
5. **Fijar un umbral** y un piloto en el que el modelo solo propone y una persona revisa.
6. **Decidir con los datos del piloto** si merece la pena seguir.

Glosario

Modelo de decisión. Modelo que, dado un texto y una pregunta con opciones cerradas, devuelve una probabilidad por opción. No genera texto.

Modelo de lenguaje (LLM). Modelo que genera texto: los chats como ChatGPT o Claude.

Calibración. Grado en que la probabilidad que da el modelo coincide con su acierto real. Bien calibrado: cuando dice 80 %, acierta 8 de cada 10.

Prompt. Texto libre con instrucciones y contenido que se le escribe a un chat. Los modelos de decisión no lo usan: reciben campos fijos (texto, pregunta, opciones).

Cabeza. Pieza pequeña que se añade a un modelo de lenguaje para que, en vez de escribir, puntúe cada opción.

Log-loss. Medida de lo bien que un modelo reparte la probabilidad: la media de lo «sorprendido» que queda con la respuesta correcta. Cuanto más bajo, mejor; castiga mucho equivocarse con seguridad.

Umbral. Probabilidad a partir de la cual se acepta la respuesta del modelo sin revisión.

Post-entrenamiento. Cualquier entrenamiento que se hace sobre un modelo ya entrenado. El ajuste fino es un caso.

Ajuste fino. Seguir entrenando un modelo ya hecho con ejemplos propios, para que responda mejor en una tarea concreta.

LoRA. Técnica de ajuste fino que congela el modelo y entrena solo unas piezas pequeñas añadidas. Reduce mucho la memoria y el tiempo necesarios.

Pesos abiertos. Modelo cuyos ficheros se publican y se pueden descargar y ejecutar en máquinas propias.

Este texto se publica con licencia Creative Commons Reconocimiento 4.0 (CC BY 4.0).